You are a Senior Machine Learning Architect specializing in Computer Vision and Sequence Modeling. I need you to design and write a hybrid neural network architecture in Python (using PyTorch or TensorFlow/Keras) for a real-time Sign Language Sentence Translation system.

The pipeline will receive coordinates from Google MediaPipe Hands and process them as sequential data.

System Specifications & Architecture Requirements:

1. INPUT DATA SHAPE
- Data Format: 3D Tensor with shape (batch_size, sequence_length, feature_dimension)
- sequence_length: 60 frames (representing a ~2-second sliding window

at 30 FPS)
- feature_dimension: 126 features (21 landmarks * 3 coordinates [x, y, z] * 2 hands)

2. HYBRID MODEL ARCHITECTURE (LSTM + Transformer)
Please implement a custom model class that sequences the following three layers:

- LAYER 1: LSTM Block (Spatio-Temporal Feature Extraction)
  * Objective: Process the frame-by-frame coordinate sequences chronologically to capture local micro-motions and smooth out MediaPipe jitter.
  * Setup: A Bidirectional LSTM layer (e.g., hidden_size=128 or 256) that outputs a dense feature vector for every time step, preserving temporal order.

- LAYER 2: Transformer Encoder Block (Global Temporal Context & Translation)
  * Objective: Accept the LSTM's motion feature vectors and apply Multi-Head Self-Attention.
  * Setup: A standard Transformer Encoder Layer (e.g., 4 attention heads). It must evaluate the mathematical relationship between all frames simultaneously to capture long-term context, phrase structures, and word transitions.

- LAYER 3: Dense Classification Head
  * Objective: Map the global context vector to the target sentence.
  * Setup: A Linear/Dense fully connected layer passing through a Softmax activation function to output class probabilities over the vocabulary size (the list of possible sentences).

What I need you to generate:
1. Provide the complete code for this custom hybrid neural network model class.
2. Include the model compilation setup showing a recommended loss function (such as Categorical Cross-Entropy) and an optimizer (such as Adam).
3. Briefly explain how the "Sliding Window" matrix buffer should feed live OpenCV/MediaPipe frames into this model's input shape during real-time inference.

Keep the code modular, production-ready, and thoroughly commented.